# Ojunai

## Engineer Onboarding Handbook

A complete map of the codebase — backend, dashboard, data & integrations, the dev/deploy workflow, and where to find everything.

Generated 2026-06-24 · WhatsApp-first AI business assistant
.NET 8 API · Next.js 15 dashboard · PostgreSQL · Hangfire

## What is Ojunai?

Ojunai is a **chat-first AI business assistant for African SMBs**. A shop owner runs their business by *talking to a bot* — on **WhatsApp, Telegram, or Facebook Messenger** — in natural language ("sold 5 bags of rice to Ade for 15k", "how much did I make today?", "spent 2k on transport"). The bot understands the message (via Claude), records the sale/expense/inventory/payment, and replies. A companion **web dashboard** gives the owner reports, settings, billing, and a full view of everything the bot recorded. There is also a separate **OjunaiVoice** product — an AI phone receptionist that answers calls.

This handbook is a map of the whole codebase: what each part does, **where to find it**, how the pieces talk to each other, the dev/deploy workflow, and a changelog of recent work. Read the Architecture section first, then dive into whichever layer you'll work on.

## System at a glance

Three deployable pieces plus external services:

- `Ojunai.API/` — the .NET 8 backend (the brain). HTTP API + the bot engine + background jobs. Talks to Postgres, the payment providers, the messaging providers, Claude, and the Voice service. Runs as a systemd service on the prod box, port 5000.
- `dashboard/` — the Next.js 15 web app (the merchant's screen). Talks only to the .NET API. Runs under PM2, port 3000.
- **PostgreSQL** — the single source of truth (`ojunai_prod`). Also backs Hangfire (background jobs).
- **Separate Voice AI service** at `voice.ojunai.com` — a standalone service the .NET API proxies to (the dashboard never calls it directly).
- **External integrations** — Paystack & Flutterwave (payments), Twilio (WhatsApp), Telegram & Messenger (chat), Anthropic Claude (NL parsing), ElevenLabs (voices), Resend (email), Cloudflare R2 (DB backups), Grafana Cloud (telemetry).

### *The core message flow (memorize this)*

The dashboard reads/writes the same data through normal REST endpoints. **Anything recorded by the bot and anything done in the dashboard hit the same services and the same database.**

### Repository layout

```
Ojunai-AI/
├── Ojunai.API/          # .NET 8 backend (API + bot engine + jobs)
│   ├── Controllers/     # HTTP endpoints (23 controllers, by domain)
│   ├── Services/        # business logic; Services/Channels/ = messaging
│   ├── Models/          # EF Core entities + enums
│   ├── DTOs/            # request/response shapes (by domain)
│   ├── Common/          # cross-cutting: BillingConfig, PlanGuard, permissions…
│   ├── Jobs/            # Hangfire recurring jobs
│   ├── Data/            # AppDbContext
│   ├── Migrations/      # EF Core migration history (auto-applied on startup)
│   ├── Middleware/      # request-context, active-user
│   └── Program.cs       # DI, middleware, Hangfire, migrations, OTel
├── dashboard/           # Next.js 15 web app
│   └── src/
│       ├── app/         # App Router pages (the (dashboard) group is auth-gated)
│       ├── components/  # shared components + ui/ primitives
│       └── lib/         # api client, data-sync, pricing, types, auth, permissions
├── scripts/             # deploy/rollback/backup bash + PDF generators
└── docs/                # runbooks (DR, observability, runbook, audits, this PDF)
```

Config files (`appsettings*.json`, `.env*`) are **gitignored and server-managed** — they hold secrets and live only on the prod box.

# Backend — Ojunai.API (.NET 8)

Layered: **Controllers** (HTTP) → **Services** (logic) → **AppDbContext / Models** (data). Cross-cutting lives in **Common/** and **Middleware/**. Background work is in **Jobs/** (Hangfire). Every controller inherits `OjunaiBaseController` (`Controllers/OjunaiBaseController.cs`), which exposes `BusinessId` and `UserId` from JWT claims — that's how multi-tenancy is enforced on every request.

### Controllers (where each route lives)

| Controller (Controllers/…) | Route prefix | What it owns |
|---|---|---|
| `AuthController.cs` | `api/auth` | register, login, logout, me, password reset/change, phone+email verification, account recovery, channel-native signup (Telegram/Messenger), DOB, alert-channel |
| `BusinessController.cs` | `api/business` | business profile, plan-status, start-trial, receipt-preview, **Voice AI settings/reservations proxy** |
| `SalesController.cs` | `api/sales` | record/list/update/void sale, receipt PDF; fires post-sale alerts |
| `InventoryController.cs` | `api/inventory` | stock-in/out/adjust, transactions; fires low-stock alerts |

| Controller (Controllers/…) | Route prefix | What it owns |
|---|---|---|
| `ProductsController.cs` | `api/products` | product catalog CRUD, low-stock list, bulk categorize |
| `ExpensesController.cs` | `api/expenses` | record/list/delete expenses |
| `ContactsController.cs` / `LedgerController.cs` | `api/contacts` / `api/ledger` | customers/suppliers; receivables/payables + payments |
| `ReportsController.cs` | `api/reports` | 25+ analytics: daily/weekly, P&L, aging, heatmap, retention, forecasts |
| `SubscriptionController.cs` | `api/subscription` | quota, pricing catalog, plan upgrade/change, WhatsApp packs, auto-renew |
| `WebhooksController.cs` | `api/webhooks` | **inbound** WhatsApp/Telegram/Messenger + Resend (signature-verified, AllowAnonymous) |
| `ChannelsController.cs` | `api/channels` | link/unlink Telegram & Messenger; channel status |
| `AlertsController.cs` | `api/alerts` | list/read/dismiss bell alerts; unread count |
| `StaffController.cs` | `api/staff` | staff CRUD + roles |
| `ImportController.cs` / `ExportController.cs` | `api/import` / `api/export` | CSV import (async job) / CSV + receipt export |
| `AdminController.cs` | `api/admin` | internal analytics/billing/observability (key-gated) |
| `DashboardController.cs`, `EventsController.cs`, `StockHoldsController.cs`, `ResendNotificationsController.cs` | various | dashboard metadata, client event logging, stock reservations, email webhooks |

Endpoints are gated with `[RequirePermission(Permission.X)]`. Auth-sensitive ones use `[AuthRateLimit]`.

## Services (the business logic)

All under `Services/`. The heavy hitters:

- `AuthService.cs` — register/login, JWT issuance, password reset, lockout, channel-native signup state machine.
- `SalesService.cs` / `InventoryService.cs` / `ProductService.cs` / `ExpenseService.cs` / `LedgerService.cs` / `ContactService.cs` — the core domain operations; validate, write entities, keep stock + ledger consistent.
- `ReportService.cs` **(+** `.Advanced.cs`**)** — 30+ analytical queries powering the dashboard and summaries.
- `ReceiptService.cs` — QuestPDF receipt rendering (custom header/footer/accent, VAT, atomic receipt number). `GeneratePreview` renders a non-persisted sample for the settings preview.
- `AlertService.cs` + `Services/Channels/NotificationDispatcher.cs` — create in-app bell alerts and route them to the user's chosen channel (WhatsApp/Telegram/Messenger) with a WhatsApp fallback for billing-critical ones.
- `PaystackService.cs` **(NGN) /** `FlutterwaveService.cs` **(other currencies)** — subscription init, webhook handling, recurring charges, mid-cycle upgrade proration.

- `UsageService.cs` — per-month quota/action counts powering quota meters.
- `ClaudeParsingService.cs` — sends the message + business context to Claude, returns structured intent + fields. Concurrency-capped by `ClaudeConcurrencyLimiter.cs`; metrics in `ClaudeMetrics.cs`.
- `WhatsAppService.cs` — the (large, legacy) WhatsApp bot engine: parse → dispatch intent → execute → reply; per-phone locks, dedup, small-talk bypass.
- `VoiceAIProvisioningService.cs` — provisions a business on the external Voice service.
- `PhoneVerificationService.cs` / `EmailVerificationService.cs` / `AccountRecoveryService.cs` / `EmailService.cs` — OTP, email, recovery.

### *Messaging architecture (Services/Channels/)*

The multi-channel abstraction lives here:

- `ConversationOrchestrator.cs` — universal inbound entry (`ProcessInboundAsync`): dedup, identity touch, then dispatch per channel (WhatsApp → legacy `WhatsAppService`; Telegram → `TelegramIntentHandler`; Messenger → `MessengerIntentHandler`).
- `ChannelRegistry.cs` — registry of `IChannelAdapter` implementations.
- `TwilioWhatsAppAdapter.cs`, `Telegram/TelegramAdapter.cs` + `TelegramIntentHandler.cs` + `TelegramSignupHandler.cs`, `Messenger/MessengerAdapter.cs` + `MessengerIntentHandler.cs` + `MessengerSignupHandler.cs` — per-channel signature verification, parsing, intent dispatch, sending.
- `NotificationDispatcher.cs` — outbound alert routing (primary channel + fallback).
- Pending multi-step flows: `PendingAction` (WhatsApp) and `Telegram/PendingTelegramActionService.cs` (Telegram/Messenger token+button flows).

There is a rollout flag `Multichannel:V1Enabled` — when off, WhatsApp takes the proven legacy path; when on, it flows through the orchestrator.

## Domain models (Models/)

Core entities — `Business` (the tenant: plan, billing, alert toggles, Voice fields, receipt settings) and `User` (role, auth, AlertChannel). Then `Sale`/`SaleItem`, `Product`, `Contact` + `ContactIdentity` (a user's identity on a channel — phone/chat_id/PSID), `Expense`, `LedgerEntry` (receivables/payables), `InventoryTransaction`, `Alert`, `BusinessAddOn` (WhatsApp packs etc.), `MessageLog`, `PendingAction`/`PendingTelegramAction`, `Subscription`/`ActionUsage` (Pricing v2). Key enums: `UserRole` (Owner/Admin/Sales/Bookkeeper/Viewer), `PaymentStatus`, `AlertType`, `AlertSeverity`, `Channel`, `Permission`, `LedgerEntryType`.

## Background jobs (Jobs/ + Hangfire)

Postgres-backed, registered in `Program.cs`, idempotent. Dashboard at `/hangfire` (localhost-only).

| Job | Schedule | Purpose |
|---|---|---|
| daily/weekly summary (`SummaryJobService`) | hourly (per-timezone) | send daily/weekly business summaries to the alert channel |

| Job | Schedule | Purpose |
|---|---|---|
| trial reminders / revert | hourly / every 4h | warn before trial ends; downgrade expired trials |
| renewal reminders (`RenewalReminderJobService`) | hourly | warn before subscription renewal |
| payment reconciliation | daily 06:00 | sync Paystack/Flutterwave charge state |
| dashboard alert generator | hourly | aged receivables, trial-ending alerts |
| whatsapp pack expiry / renewal reminder | daily 03:00 / 03:30 | expire/remind WhatsApp packs |
| Voice AI trial revert | every 4h | expire Voice trials |
| admin daily snapshot | 00:05 | DAU/WAU/MAU/MRR → snapshot table |
| message-log retention | daily 02:00 | delete message logs > 180 days |
| import processor (`ImportJobService`) | on demand | async CSV import |

## Cross-cutting (Common/, Middleware/, Program.cs)

- **Auth & permissions** — JWT (cookie or bearer). `[RequirePermission(Permission.X)]` checks `User.Role` against the `RolePermissions` matrix → 403 if denied. `Middleware/RequestContextMiddleware.cs` puts BusinessId/UserId into the logging scope; `ActiveUserMiddleware.cs` blocks suspended users.
- `Common/BillingConfig.cs` — the **pricing source of truth** (plan × cycle × currency matrix, WhatsApp pack prices, currency → provider routing). The dashboard reads this live via `GET /subscription/pricing`.
- `Common/AlertChannels.cs` — the `"none"`/whatsapp/telegram/messenger constants + the rule that business alerts only fire once a channel is chosen.
- `Common/PlanGuard.cs` / `PlanLimits.cs` / `VoiceAIGuard.cs` — trial/subscription state + feature gating.
- `Common/ApiResponse.cs` — every endpoint returns `{ success, message?, data?, errors? }`. Lists use `PaginatedResult<T>`.
- `Program.cs` — DI registration, named HttpClients (Claude/Paystack/Flutterwave/VoiceAI), Hangfire server + recurring jobs, the global exception handler (maps `KeyNotFoundException`/`InvalidOperationException`/`ArgumentException` → 400), `MigrateAsync()` **on startup**, and OpenTelemetry.

# Frontend — dashboard (Next.js 15)

Stack: **Next.js 15 App Router**, **React 19**, **TypeScript**, **Tailwind**, **TanStack React Query** (server state), **Axios** (HTTP). Path alias `@/*` → `src/*`. Pages are mostly client components (`"use client"`). It talks **only** to the .NET API.

## Routing & layout

- `src/app/(dashboard)/layout.tsx` — the auth-gated shell: redirects unauthenticated users to `/login`, wraps children in `<DataSyncProvider>` (business/user context), renders the sidebar + banners.
- Public routes live outside the group: `login`, `register`, `forgot-password`, `change-password`, `recover`, `verify-email`, `post-signup`, `install`, `offline`, `privacy`, `terms`.
- `src/app/providers.tsx` — QueryClientProvider, ThemeProvider, Toaster, CommandPalette.

## Pages (src/app/(dashboard)/…)

| Route | File | Purpose |
|---|---|---|
| `/` | `page.tsx` | Today: sales/expenses/low-stock/trends |
| `/sales` | `sales/page.tsx` | record & manage sales |
| `/inventory` | `inventory/page.tsx` | products + stock |
| `/expenses` | `expenses/page.tsx` | expenses |
| `/reports` | `reports/page.tsx` | analytics |
| `/contacts` | `contacts/page.tsx` | customers/suppliers + ledger |
| `/reservations` | `reservations/page.tsx` | Voice-AI bookings |
| `/activity` | `activity/page.tsx` | event log |
| `/settings` | `settings/page.tsx` | profile, billing, **Chat Channels**, alerts, receipts |
| `/voice-ai` | `voice-ai/page.tsx` | OjunaiVoice config + usage |
| `/import`, `/export`, `/get-started` | … | bulk import / export / onboarding |

Internal `src/app/admin/` pages (overview, analytics, telemetry, voice-ai) are staff-only.

## Components (src/components/)

Shared: `sidebar.tsx`, `page-header.tsx`, `command-palette.tsx` (⌘K), `notification-bell.tsx` (polls unread alerts), `quota-meter.tsx`, `trial-banner.tsx`, `whatsapp-pack-picker.tsx`, `install-*` (PWA), `toast.tsx`, `settings-nav.tsx` / `settings-section.tsx`. Primitives in `src/components/ui/` (shadcn-style): `button`, `input`, `password-input`, `card`, `dialog`, `drawer`, `select`, `badge`, `table`, `tabs`, `separator`, `skeleton`.

## lib/ — the plumbing every page uses

- `src/lib/api.ts` — the Axios instance: base `NEXT_PUBLIC_API_URL`, `withCredentials` (JWT cookie), a 401 interceptor that logs out + redirects. **Every API call goes through this.**
- `src/lib/data-sync.tsx` — `<DataSyncProvider>` + `useBusiness()` / `useUser()` / `useDataSync()`. Loads business+user from localStorage then refreshes from `/business` + `/auth/me`. Pages read business/user from here; after a settings save you call `refresh()`.
- `src/lib/use-pricing.ts` — `usePricing()` (tiers from `/subscription/pricing`) and `useVoicePricing()` (`/subscription/voice-ai-pricing`). **Prices are not hardcoded** — they come live from the backend.
- `src/lib/pricing.ts` — currencies, `formatPrice`, `getProvider`, `toBillingCurrency` (no prices).
- `src/lib/types.ts` — all DTO interfaces + the `ApiResponse<T>` envelope.
- `src/lib/permissions.ts` — `hasPermission(Permission.X)` (client-side gate; server re-checks).
- Others: `auth.ts`, `format.ts`, `phone.ts`, `password-policy.ts`, `alerts.ts`, `pwa.ts`, `utils.ts` (`cn`).

## Conventions to copy

- **Fetch:** `useQuery({ queryKey:[…], queryFn: () => api.get(...).then(r => r.data.data) })`. Include filters in the query key.
- **Save:** `await api.put/patch(...)` then either `queryClient.invalidateQueries(...)` or `refresh()` from data-sync, then a toast.
- **Envelope:** most endpoints return `{ data: T }` — unwrap `res.data.data`. A few (pricing) return the object raw; handle both.
- **Auth:** JWT is an HTTP-only cookie; only a PII-stripped user subset is cached in localStorage.

# Data, integrations & config

## Database & migrations

- **PostgreSQL** `ojunai_prod` (role `ojunai_db`). EF Core via Npgsql. Context: `Ojunai.API/Data/AppDbContext.cs`. Hangfire tables live in the same DB.
- **Migrations auto-apply on API startup** — `Program.cs` calls `db.Database.MigrateAsync()` on boot (failure is logged, not fatal). So **deploying a new API build applies any pending migration on restart.** The deploy *scripts* don't run migrations; the *app* does.
- **Add one:** in `Ojunai.API/`, `dotnet ef migrations add <Name>`, inspect the file, commit it alongside the model change. A pre-commit hook runs `dotnet ef migrations has-pending-model-changes` to stop you adding a field without a migration. Verify after deploy that it actually applied.

## External integrations & their config keys

The .NET API holds all secrets (server-side `appsettings`). The dashboard never sees them.

| Integration | Config keys | Wired in | Inbound webhook |
|---|---|---|---|
| Paystack (NGN) | `Paystack:SecretKey` | `Services/PaystackService.cs` | `/api/webhooks/paystack` |
| Flutterwave (others) | `Flutterwave:SecretKey` | `Services/FlutterwaveService.cs` | `/api/webhooks/flutterwave` |
| Twilio WhatsApp | `Twilio:AccountSid/AuthToken/WhatsAppFrom` | `Services/Channels/TwilioWhatsAppAdapter.cs` | `/api/webhooks/whatsapp` |
| Telegram | `Telegram:BotToken/WebhookSecret/BotUsername` | `Services/Channels/Telegram/` | `/api/webhooks/telegram` |
| Messenger | `Messenger:AppSecret/PageAccessToken/PageId/VerifyToken` | `Services/Channels/Messenger/` | `/api/webhooks/messenger` |
| Claude (Anthropic) | `Claude:ApiKey/Model/MaxConcurrency` | `Services/ClaudeParsingService.cs` | — (outbound) |
| Voice AI service | `VoiceAI:BaseUrl` (`voice.ojunai.com`), `VoiceAI:VoiceAdminKey` | `BusinessController.cs` proxy + `VoiceAIProvisioningService.cs` | — (outbound) |
| Email (Resend) | `Email:Smtp*`, `Email:ResendWebhookSecret` | `Services/EmailService.cs` | `/api/webhooks/resend` |
| Telemetry (Grafana) | `OTEL_EXPORTER_OTLP_ENDPOINT/HEADERS` | `Program.cs` | — (export) |

**Voice AI is a separate service.** The dashboard calls `/business/voice-ai-settings` on the .NET API, which **proxies the request verbatim** (with `X-Admin-Key`) to `voice.ojunai.com`. Never put the admin key or the voice URL in the browser.

## Config & secrets

`appsettings.json` / `appsettings.Production.json` are **gitignored** and live on the server (`/var/www/ojunai-api/`). Read via `IConfiguration["Section:Key"]`. Required keys (DB connection string, `Jwt:Secret`, `Twilio:AccountSid`) are validated at startup. Secret rotation is documented in `docs/production-secrets-rotation-plan.md`.

# Dev & deploy workflow

## Local setup

```
git clone <repo> && cd Ojunai-AI
./scripts/install-hooks.sh                  # EF migration guard pre-commit hook
cd Ojunai.API && dotnet restore              # create appsettings.Development.json (local DB + test keys)
dotnet ef database update && dotnet run      # API on :5000
cd ../dashboard && npm install               # set NEXT_PUBLIC_API_URL=http://localhost:5000/api
npm run dev                                  # dashboard on :3000
```

## Deploy (scripts/)

Both scripts build locally first (to catch errors), back up the current server build, ship, and restart. They SSH to `bizpilot@46.225.108.35` and use `sudo` (interactive) — run them from your machine.

- `./scripts/deploy-api.sh` — `dotnet publish` → scp to `/var/www/ojunai-api` → `systemctl restart ojunai-api` → poll `/health`. **Migrations auto-apply on this restart.**
- `./scripts/deploy-dashboard.sh` — local `npm run build` → upload src → `npm ci` + build on server → `pm2 restart ojunai-dashboard`.
- `./scripts/rollback-api.sh` / `rollback-dashboard.sh` — restore the previous timestamped backup.

## Server operations (quick reference)

```
ssh bizpilot@46.225.108.35
sudo systemctl status ojunai-api          # API service
sudo journalctl -u ojunai-api -f          # API logs
pm2 status ojunai-dashboard               # dashboard
sudo -u postgres psql ojunai_prod         # DB (no password, peer auth) — schema checks
curl http://localhost:5000/health         # API health
# /hangfire dashboard is localhost-only
```

## Backups & DR

`scripts/backup-db.sh` runs nightly (cron, root): `pg_dump` of `ojunai_prod` **off-box to Cloudflare R2** (S3-compatible). Local 7-day retention; off-box keeps history. Restore steps + R2 setup: `docs/disaster-recovery.md`.

# Where to find X (quick index)

| I need to… | Look here |
|---|---|
| Add/change an API endpoint | `Ojunai.API/Controllers/<Domain>Controller.cs` |
| Change business logic | `Ojunai.API/Services/<Domain>Service.cs` |
| Add a DB field | `Ojunai.API/Models/<Entity>.cs` → `dotnet ef migrations add …` |
| Change prices / plans | `Ojunai.API/Common/BillingConfig.cs` (dashboard reads it live) |
| Touch the bot's understanding | `Ojunai.API/Services/ClaudeParsingService.cs` |
| WhatsApp bot behavior | `Ojunai.API/Services/WhatsAppService.cs` |
| Telegram/Messenger behavior | `Ojunai.API/Services/Channels/{Telegram,Messenger}/…IntentHandler.cs` |
| Inbound webhook handling | `Ojunai.API/Controllers/WebhooksController.cs` |
| Background/scheduled job | `Ojunai.API/Jobs/` + the registrations in `Program.cs` |
| Permissions / roles | `Ojunai.API/Common/RolePermissions.cs` + `[RequirePermission]` |
| Receipt PDF | `Ojunai.API/Services/ReceiptService.cs` |
| Payments | `Ojunai.API/Services/{Paystack,Flutterwave}Service.cs` |
| Add a dashboard page | `dashboard/src/app/(dashboard)/<route>/page.tsx` |
| Shared dashboard logic | `dashboard/src/lib/` (api, data-sync, use-pricing, types) |
| A UI primitive | `dashboard/src/components/ui/` |
| Deploy / rollback | `scripts/deploy-*.sh`, `scripts/rollback-*.sh` |
| Runbooks / DR / audits | `docs/` |

# Recent major work (changelog)

The work done across the latest sessions, so you know what changed and why.

## Billing & pricing

- **Single source of truth for prices.** The dashboard no longer hardcodes tier prices; `usePricing()`/`useVoicePricing()` fetch them live from the backend (`/subscription/pricing`). Removed the stale `pricing.ts` table that had drifted from `BillingConfig.cs`.
- **Flutterwave fixes:** stopped truncating recurring-plan amounts with an `(int)` cast (USD/GBP were under-charged); added **mid-cycle delta upgrade** proration mirroring Paystack (charge only the difference, keep the cycle end); tightened the webhook amount tolerance.
- **WhatsApp packs** now follow the Plan & Billing currency (one section-level dropdown), with clearer "actions/month" and one-time-vs-auto-renew copy.

## Alerts & channels

- `User.AlertChannel` **defaults to** `"none"` — business alerts (low stock, daily summary, large sale) only fire once a channel is selected; billing alerts keep a WhatsApp safety net. Gating added across the summary job and the real-time alert paths.
- **Per-channel large-sale confirmation** wired into the Telegram & Messenger bots (new `Business.ConfirmLargeSales{Telegram,Messenger}` fields + a confirm→reply→execute flow; migration `AddPerChannelSaleConfirmation`).
- **Settings "Chat Channels"** — consolidated the separate WhatsApp/Telegram/Messenger integration sections + Connected Channels into one section; per-channel sale-confirm toggles inline; examples shown once.

## Receipts, account, voice

- **Receipt preview** — a "Preview receipt" button renders a real sample PDF (via `ReceiptService.GeneratePreview` + `POST /business/receipt-preview`) in an in-app overlay, reflecting unsaved settings; no Sale created.
- **Birth-year change** emits a personal bell alert (`AlertType.ProfileUpdated`, value kept private) + a success toast.
- **Password reset hardened** — unknown numbers silently no-op (kills log spam + closes account enumeration).
- **OjunaiVoice settings UI** — multilingual (en/yo/ha/ig/fr/es/zh), single `greetingTemplate`, per-language ElevenLabs voices, streaming transport with a confirm modal; regrouped into General/Voice/Calls; diff-based PATCH through the existing server-side proxy.
- **Auth UX** — password show/hide toggle + an all-four-character-class password policy (client + server).

## Foundations (earlier)

- **Database backups** added (were missing) — nightly off-box to Cloudflare R2 (`scripts/backup-db.sh`, `docs/disaster-recovery.md`).
- **Observability** — OpenTelemetry → Grafana Cloud (`docs/observability-setup.md`).

- **Scalability audit** (`docs/scalability-audit-2026-06.md`) — pre-go-live 100× review (GO-WITH-CONDITIONS); known P1s: inbound idempotency race, billing activation race, unbounded Claude concurrency, daily-summary recompute waste.

# New-engineer checklist

- Read **Architecture** + the **core message flow** above.
- Clone, run `./scripts/install-hooks.sh`, get `appsettings.Development.json` + dashboard `.env.local` from the team, run both apps locally.
- Skim `docs/design-principles.md`, then `docs/runbook.md`.
- Trace one real path end-to-end: an inbound WhatsApp "sold 2 rice for 5k" → `WebhooksController` → orchestrator → Claude → `SalesService` → DB → reply. Then the same sale via `POST /api/sales` from the dashboard.
- Bookmark: `Program.cs`, `AppDbContext.cs`, `BillingConfig.cs`, `ClaudeParsingService.cs`, the `*Adapter.cs` files, and `dashboard/src/lib/api.ts` + `data-sync.tsx`.
- Get SSH access to prod; learn `journalctl -u ojunai-api -f` and `sudo -u postgres psql ojunai_prod`.
- Remember: **prices live in `BillingConfig.cs`**, **migrations auto-apply on API restart**, **secrets are server-side**, and **the Voice service is reached only through the .NET proxy**.